Time Varying Parameters Pt.2

Ram Prasad

4/25/2025

Continuation of TVP exercise with more estimation methods and a different simulation setup.

Standard imports.

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
import matplotlib.pyplot as plt
import pickle
import time
import seaborn as sns

from scipy import stats
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_percentage_error
```

Define functions for regression and error metrics.

```
#Kernel regression
def kernel_reg(Y, X, h=None, plot=False, kernel= lambda x:((2*np.pi)**-0.5)*np.exp(-0.5*(x**2)) ):
    """
    Perform Kernel Regression.
    Args:
        Y: array like, vector of response observations
        X: array like, vector of covariate observations
        h, float: bandwidth for Gaussian kernel.
        plot, boolean: flag to generate plots
        kernel, function: kernel function. Defaults to Gaussian.
    Returns:
        kernel_results, pd.DataFrame: DF containing parameter estimates, CIs, p-values, R2, MAPE, point APE
    Default kernel bandwidth is T^0.5. Literature suggests T^0.5 or T^0.4.
    """
    #Check for valid inputs
    if len(Y) != len(X):
        raise ValueError(f"Response vector Y length {len(Y)}, differs from covariate vector length {len(X)}.")




    #Reshape, prepend column of 1s to X
    Y = np.array(Y).reshape(len(Y),1)
    X = np.array(X).reshape(len(X),1)
    X = np.hstack((np.ones((len(X),1)), X))

    #Default behavior for kernel bandwidth
    if h is None:
        #Set BW = T^0.5
        h = len(Y)**0.5


    #Define index for kernel function
    i = np.arange(1,len(Y)+1)

    #WLS est. W is a diagonal matrix with weight for obs i, w_i, at position (i,i) and 0s elsewhere
    def beta_wls(X, W, Y): return np.linalg.inv(X.T @ W @ X) @ (X.T @ W @ Y)

    #WLS SE and P-values. MSE is estimate of unknown variance sigma^2
    def cov_wls(X, W, Y, mse):
        return mse*(np.linalg.inv(X.T @ W @ X) @ (X.T @ W.T @ W @ X) @ np.linalg.inv(X.T @ W @ X))

    #per t, estimate beta, SE, p-val, R2, and point APE
    kernel_results = []
    for t in range(1, len(Y)+1):

        #compute weights
        w_t = kernel((t-i)/h)

        #W is a diagonal matrix with w_i at (i,i)
        W = np.diag(w_t)

        #Compute beta estimates
        try:
            beta = beta_wls(X, W, Y)
            b0, b1 = beta[0][0], beta[1][0]

            #Compute SSE, MSE, R2, MAPE, and point APE
            Y_pred = X @ beta
            sse = (Y - Y_pred).T @ W @ (Y - Y_pred)
            mse = sse/(len(Y) - 2 - 1)
            r2 = 1 - np.sum((Y - Y_pred)**2)/np.sum((Y - np.mean(Y))**2)
            mape = 100.0*np.mean(np.abs(Y - Y_pred)/np.abs(Y))
            t_ape = 100.0*np.abs(Y[t-1] - Y_pred[t-1])/np.abs(Y[t-1])[0]
```

```
        #Compute covariance of beta
        cov_beta = cov_wls(X,W, Y, mse)

        #Extract standard errors
        se_b0 = np.sqrt(cov_beta[0][0])
        se_b1 = np.sqrt(cov_beta[1][1])

        #Construct CIs, 5% two sided, using t dist w/ T-p-1 DF.
        ci_b0_l = b0 - 1.9615*se_b0
        ci_b0_u = b0 + 1.9615*se_b0
        ci_b1_l = b1 - 1.9615*se_b1
        ci_b1_u = b1 + 1.9615*se_b1

        #p-value, (beta_hat - 0) / se ~ t(n-p-1)
        p_b0 = stats.t.sf(np.abs(b0/se_b0), len(Y)-2-1)*2
        p_b1 = stats.t.sf(np.abs(b1/se_b1), len(Y)-2-1)*2
    except np.linalg.LinAlgError as l:
        """
        Error at last value of t using custom kernel, because only (t,t) element of
        #weight matrix is non-zero. All else are 0, so reg. tries using 1 obs w/intercept
        and covariate hence Linear Algebra error
        """
        print(f"Linear Algebra Error: {l} at t={t}.")

        #If so, set all values to nan
        ci_b0_l, b0, ci_b0_u, p_b0 = np.nan, np.nan, np.nan, np.nan
        ci_b1_l, b1, ci_b1_u, p_b1 = np.nan, np.nan, np.nan, np.nan
        r2, t_ape, mape = np.nan, np.nan, np.nan

    #This segement executes regardless of error
    else:
        #Track results
        t_results = {"t":t,"CI.L.B0":ci_b0_l, "B0":b0, "CI.U.B0":ci_b0_u,"P(B0)":p_b0,
                     "CI.L.B1":ci_b1_l, "B1":b1, "CI.U.B1":ci_b1_u,"P(B1)":p_b1,
                 "R2":r2, "APE":t_ape, "MAPE":mape}
        kernel_results.append(t_results)

#Store as DF
kernel_results = pd.DataFrame.from_dict(kernel_results)

#Plot
if plot:
    #Plot kernel estimates, CIs, p-values
    fig, axs = plt.subplots(nrows=3, ncols=1, figsize=(15,10))
```

```
    #Beta 0 and 1
    for b in [0,1]:

        axs[b].plot(kernel_results.t, kernel_results[f"B{b}"], label=r"$\hat{\beta}$"+str(b))
        axs[b].fill_between(x=kernel_results.t,
                            y1=kernel_results[f"CI.L.B{b}"],
                            y2=kernel_results[f"CI.U.B{b}"], label="95% CI", alpha=0.2)

        #Overlay p-values
        axs[b].scatter(kernel_results.t,
                       np.where(kernel_results[f"P(B{b})"]<0.05, kernel_results[f"B{b}"], np.nan),
                       color="blue", label="p<5%", s=5)

        if b==0:
            axs[b].set_title("const")
        else:
            axs[b].set_title("x1")
        axs[b].grid()
        axs[b].legend()
        axs[b].axhline(y=0, linestyle="--", color="black", alpha=0.5)

    #Plot R2
    axs[2].plot(kernel_results.t, kernel_results.R2, color="red")
    axs[2].set_title(r"$R^{2}$")
    axs[2].grid()

    #Set title, x label, and display
    plt.suptitle(f"Kernel Regression Results, T={len(Y)}")
    plt.xlabel(f"t")
    plt.tight_layout()
    plt.show()

    #Separate plots for MAPE and APE
    fig, axs = plt.subplots(nrows=2, ncols=1, figsize=(15,10))
    axs[0].plot(kernel_results.t, kernel_results.MAPE, color="red")
    axs[0].set_title(r"$MAPE(t)$")
    axs[0].set_ylabel("%")
    axs[0].grid()
    axs[1].plot(kernel_results.t, kernel_results.APE, color="red", linestyle="--")
    axs[1].set_title(r"$APE(t)$")
    axs[1].set_ylabel("%")
    axs[1].grid()
    plt.suptitle(f"Kernel Regression Results, T={len(Y)}")
    plt.xlabel(f"t")
    plt.tight_layout()
```

```
        plt.show()

    #Return dataframe
    return kernel_results


#Kernel regression. Shorter function for simulation.
def kernel_reg_sim(Y, X, h=None, kernel= lambda x:((2*np.pi)**-0.5)*np.exp(-0.5*(x**2)) ):
    """
    Perform Kernel Regression.
    Args:
        Y: array like, vector of response observations
        X: array like, vector of covariate observations
        h, float: bandwidth for Gaussian kernel.
        kernel, function: kernel function. Defaults to Gaussian.
    Returns:
        kernel_results, pd.DataFrame: DF containing parameter estimates, CIs, p-values, R2, MAPE, point APE
    Default kernel bandwidth is T^0.5. Literature suggests T^0.5 or T^0.4.
    """
    #Check for valid inputs
    if len(Y) != len(X):
        raise ValueError(f"Response vector Y length {len(Y)}, differs from covariate vector length {len(X)}.")

    #Reshape, prepend column of 1s to X
    Y = np.array(Y).reshape(len(Y),1)
    X = np.array(X).reshape(len(X),1)
    X = np.hstack((np.ones((len(X),1)), X))

    #Default behavior for kernel bandwidth
    if h is None:
        #Set BW = T^0.5
        h = len(Y)**0.5


    #Define index for kernel function
    i = np.arange(1,len(Y)+1)

    #WLS est. W is a diagonal matrix with weight for obs i, w_i, at position (i,i) and 0s elsewhere
    def beta_wls(X, W, Y): return np.linalg.inv(X.T @ W @ X) @ (X.T @ W @ Y)

    #WLS SE and P-values. MSE is estimate of unknown variance sigma^2
    def cov_wls(X, W, Y, mse):
        return mse*(np.linalg.inv(X.T @ W @ X) @ (X.T @ W.T @ W @ X) @ np.linalg.inv(X.T @ W @ X))





    #per t, estimate beta
    kernel_results = []
    for t in range(1, len(Y)+1):

        #compute weights
        w_t = kernel((t-i)/h)

        #W is a diagonal matrix with w_i at (i,i)
        W = np.diag(w_t)

        #Compute beta estimates
        try:
            beta = beta_wls(X, W, Y)
            b0, b1 = beta[0][0], beta[1][0]

            #Compute SSE, MSE, R2, MAPE, and point APE
            Y_pred = X @ beta
        except np.linalg.LinAlgError as l:
            """
            Error at last value of t using custom kernel, because only (t,t) element of
            #weight matrix is non-zero. All else are 0, so reg. tries using 1 obs w/intercept
            and covariate hence Linear Algebra error
            """
            #If so, set all values to nan
            b0 = np.nan
            b1 = np.nan

        #This segement executes regardless of error
        else:
            #Track results
            t_results = { "B0":b0,"B1":b1}
            kernel_results.append(t_results)

    #Store as DF
    kernel_results = pd.DataFrame.from_dict(kernel_results)

    #Return dataframe
    return kernel_results

#Utility function for error metrics
def error_metrics(y_pred, y_true, label, plot=True):
    """
    Compute R2, MSE, MAPE. Plot.
    Args:
        y_pred: array, vector of predicted observations.
```

```
        y_true: array, vector of true observations.
        label: str, name of prediction method
        plot: boolean, flag to generate plots of true vs. predicted
    Returns:
        metrics: pd.DataFrame
    Use sklearn error measures. Plot residuals, co-movement, scatter.
    ACF of residuals and residuals squared for GARCH.
    """
    #Check same length
    try:
        assert(len(y_pred)==len(y_true))
    except AssertionError:
        print(f"Pred. length {len(y_pred)} does not equal actual length {len(y_true)}.")
        return
    #R2
    r2 = r2_score(y_true, y_pred)

    #RMSE
    rmse = mean_squared_error(y_true, y_pred)**0.5

    #MAPE
    mape = mean_absolute_percentage_error(y_true, y_pred)

    #Correlation
    r = np.corrcoef(y_pred, y_true)[0][1]

    #compute residuals
    resid = y_true - y_pred

    #Plot
    if plot:
        #4 subplots
        fig, axs = plt.subplots(nrows=3, ncols=2, figsize=(15,10))

        #Actuals vs. predicted
        axs[0,0].plot(y_true, label="Actual", linestyle="--")
        axs[0,0].plot(y_pred, label="Predicted", linestyle="--")
        axs[0,0].set_title("Actual vs. Predicted:"+r'$\rho$='+str(np.round(r, 2)))
        axs[0,0].grid()
        axs[0,0].legend()
        axs[0,0].set_xlabel("T")

        #Time series of residuals
        axs[0,1].plot(resid)
        axs[0,1].set_title("Residuals")




        axs[0,1].grid()
        axs[0,1].set_xlabel("T")

        #ACF residuals.
        sm.graphics.tsa.plot_acf(resid, lags=40, ax=axs[1,0])
        axs[1,0].grid()
        axs[1,0].set_title("Resid. ACF")

        #ACF of squared resid for ARCH effect.
        sm.graphics.tsa.plot_acf(resid**2, lags=40, ax=axs[1,1])
        axs[1,1].grid()
        axs[1,1].set_title("Resid. Sq. ACF")

        #Histogram residuals
        axs[2,0].hist(resid)
        axs[2,0].set_title("Resid. Dist. (e~N(0,1))")
        axs[2,0].grid()
        axs[2,0].axvline(x=np.mean(resid),
                         label=f"{np.round(np.mean(resid),2)}",
                         color="red", linestyle="--")
        axs[2,0].legend()


        #Scatter residuals
        axs[2,1].scatter(y_true, y_pred)
        axs[2,1].grid()
        axs[2,1].set_xlabel("Actual")
        axs[2,1].set_ylabel("Pred.")

        #Set title and display
        plt.suptitle(f"{label} Diagnostic Plots")
        plt.tight_layout()
        plt.show()

    #store results as DF
    metrics = {"R2":r2,"RMSE":rmse,"MAPE":mape,"Corr.":r,"T":len(y_true), "Sim:":label}
    return metrics


#Utility function to simulate grid of Y(t) with time-varying parameters
def sim_tvp(X, n=1000, T=1000, burn_in=0.20, mu_a=0.0, mu_b=0.0, phi_a = 0.99, phi_b=0.99,
            sigma_e=1.0, sigma_a=1.0, sigma_b=1.0):
    """
    Simulate the process Y(t) = a(t) + b(t)*X(t) + e(t), e(t) ~ WN(0, sigma_e^2)
    where a(t) and b(t) are AR(1) processes:
```

```
    a(t) = mu_a + phi_a*a(t-1) + w_a(t), w_a(t) ~ WN(0, sigma_a^2)
    b(t) = mu_b + phi_b*b(t-1) + w_b(t), w_b(t) ~ WN(0, sigma_b^2)
    Args:
        n: int, number of simulations
        T: int, length of simulations
        burn_in: float, % of T to use as burn in simulated values
        mu_a: float, the drift term for a(t)'s AR(1)
        mu_b: float, the drift term for b(t)'s AR(1)
        phi_a: float, a(t)'s autoregressive coefficient
        phi_b: float, b(t)'s autoregressive coefficient
        sigma_e, sigma_a, sigma_b: float, white noise volatility for Y(t), a(t), and b(t), respectively
        X: array like, a vector of coefficients
    Returns:
        Y_sim: np.array, an n x T of simulated values for Y(t)
        a_sim: np.array, an n x T of simulated values for a(t)
        b_sim: np.array, an n x T of simulated values for b(t)
    White noise processes are serially and mutually uncorrelated. Use Gaussian WN. AR(1) coefficients must
    be within [-1,1].
    """
    #Check input phi values are valid: -1.0 <= phi <= 1.0
    if np.abs(phi_a) > 1:
        raise ValueError(f"phi_a: {phi_a}, must be between -1 and 1 inclusive.")
    if np.abs(phi_b) > 1:
        raise ValueError(f"phi_b: {phi_b}, must be between -1 and 1 inclusive.")

    #Check WN volatility is non-negative
    if not (sigma_e > 0.0 and sigma_a > 0.0 and sigma_b > 0.0):
        raise ValueError(f"White noise volatility parameters are {sigma_e, sigma_a, sigma_b}, must be strictly positive."

    #Check X_t is same length as T
    if len(X) != T:
        raise ValueError(f"Covariate vector X(t) has length {len(X)}, must equal {T}.")

    #Set length s.t. after discarding burn in, final series length T
    sim_length = int(np.round((1+burn_in)*T,0))

    #Initialize array of 0s of dim n x T to hold simulations
    Y_sim = np.zeros((n, T))

    #Initalize arrays to hold simulated a_t and b_t
    a_sim = np.zeros((n,T))
    b_sim = np.zeros((n,T))

    #Construct Y_t, a_t and b_t
    for sim in range(n):


        #Initialize vectors for a_t, b_t. Initial values are drift terms.
        a = np.zeros(sim_length)
        a[0] = mu_a
        b = np.zeros(sim_length)
        b[0] = mu_b

        #Generate randomw draws for WN
        e = np.random.normal(0, sigma_e**2, sim_length)
        wa = np.random.normal(0, sigma_a**2, sim_length)
        wb = np.random.normal(0, sigma_b**2, sim_length)

        #Construct a_t, b_t
        for t in range(1, sim_length):
            a[t] = mu_a + phi_a*a[t-1] + wa[t]
            b[t] = mu_b + phi_b*b[t-1] + wb[t]

        #Discard burn-in obs
        a, b, e = a[-T:], b[-T:], e[-T:]

        #Construct Y(t). Use element by element multiplication.
        Y = a + b*X + e

        #Update matrix
        Y_sim[sim, :] = Y
        a_sim[sim, :] = a
        b_sim[sim, :] = b

    #Return matrix of simulations, each row is a simulation
    return Y_sim, a_sim, b_sim


#Rolling OLS
def rolling_ols(Y, X, w, plot=True):
    """
    Perform rolling OLS estimation.
    Args:
        y, array: vector of response variables
        x, array: vector of covariates
        w, int: window size
        plot, boolean: flag to display rolling estimates
    Returns:
        rolling_results, pd.DataFrame: dataframe containing rolling estimates, CIs, p-values, R2
    Regress y on x and a constant using data in window of size w. Use NW standard errors for p-vals.
    Y(t) = B0 + B1*X(t) + e(t)
```

```
    ---
    #Check valid inputs
    if len(Y) != len(X):
        raise ValueError(f"Response vector Y length {len(Y)}, differs from covariate vector length {len(X)}.")
    if w >= len(Y):
        raise ValueError(f"Window size {w} too large.")

    #Collect rolling results.
    rolling_results = []

    #Roll through data, regress y on x with a constant, retain param est, R2, SEs, p-value
    for i in range(0,len(Y)-w+1):

        #Subset data
        Y_w = Y[i:i+(w-1)]
        X_w = sm.add_constant(X[i:i+(w-1)])

        #Store index range (inclusive, exclusive)
        idx_min, idx_max = i, i+(w-1)

        #Regress
        nw_lag = int(len(X_w)**0.25)
        w_model = sm.OLS(Y_w, X_w).fit(cov_type="HAC", cov_kwds={"maxlags":nw_lag})

        #CI for beta 0
        ci_b0_u = w_model.conf_int().T["const"][1]
        ci_b0_l = w_model.conf_int().T["const"][0]

        #CI for beta 1
        ci_b1_u = w_model.conf_int().T["x1"][1]
        ci_b1_l = w_model.conf_int().T["x1"][0]

        #Parameter estimates and p values
        b0_w = w_model.params["const"]
        p0_w = w_model.pvalues["const"]
        b1_w = w_model.params["x1"]
        p1_w = w_model.pvalues["x1"]

        #R2
        r2_w = w_model.rsquared

        #Store results
        w_results = {"Min.":idx_min, "Max.":idx_max, "CI.L.B0":ci_b0_l, "B0":b0_w, "CI.U.B0":ci_b0_u,"P(B0)":p0_w,
                                                     "CI.L.B1":ci_b1_l, "B1":b1_w, "CI.U.B1":ci_b1_u,"P(B1)":p1_w,
                     "R2":r2_w}




        rolling_results.append(w_results)

    #Store results as DF
    rolling_results = pd.DataFrame(data = rolling_results)

    if plot:
        #Plot rolling estimates, CIs, p-values
        fig, axs = plt.subplots(nrows=3, ncols=1, figsize=(15,10))

        #Beta 0 and 1
        for b in [0,1]:

            axs[b].plot(rolling_results.index, rolling_results[f"B{b}"], label=r"$\hat{\beta}$"+str(b))
            axs[b].fill_between(x=rolling_results.index,
                                y1=rolling_results[f"CI.L.B{b}"],
                                y2=rolling_results[f"CI.U.B{b}"], label="95% CI", alpha=0.2)

            #Overlay p-values
            axs[b].scatter(rolling_results.index,
                           np.where(rolling_results[f"P(B{b})"]<0.05, rolling_results[f"B{b}"], np.nan),
                           color="blue", label="p<5%", s=5)

            axs[b].set_title(w_model.model.exog_names[b])
            axs[b].grid()
            axs[b].legend()
            axs[b].axhline(y=0, linestyle="--", color="black", alpha=0.5)

        #Plot rolling R2
        axs[2].plot(rolling_results.index, rolling_results.R2, color="red")
        axs[2].set_title(r"$R^{2}$")
        axs[2].grid()

        #Set title, x label, and display
        plt.suptitle(f"Rolling OLS Results, W={w}, T={len(Y)}")
        plt.xlabel(f"Window End\nNewey-West Std. Errors, {nw_lag} max. lags")
        plt.tight_layout()
        plt.show()

    return rolling_results


def sequential_ols(Y, X, plot=True):
    """
    Perform sequential OLS. Incrementally expand the subset of observations over
```

```
which the regression is fit, beginning with the minimum (k+1 for k regressors)
and ending (inclusive) at T.
Args:
    Y, array: a vector of response observations
    X, array: a vector of covariates
    plot, boolean: a flag for diagnostic plots
Returns:
    sequential_results, pd.DataFrame(): a DF containing parameter estimates, CIs, p-values, R2
"""
#Check for valid inputs
if len(Y) != len(X):
    raise ValueError(f"Response vector Y length {len(Y)}, differs from covariate vector length {len(X)}.")

#Reshape, prepend column of 1s to X
data = pd.DataFrame()
data["Y"] = Y
data["x1"] = X

#Initialize empty list to hold results
sequential_results = []

#Expand window, CG 2015 regression, retain param est, R2, SEs, p-value
for i in range(2+1,len(Y)+1):

    #Subset and regress
    Y_s = data.iloc[0:i]["Y"]
    X_s = sm.add_constant(data.iloc[0:i]["x1"])
    nw_lag = int(len(X_s)**0.25)
    s_model = sm.OLS(Y_s, X_s).fit(cov_type="HAC", cov_kwds={"maxlags":nw_lag})

    #window size (portion)
    w_s = len(Y_s)/len(Y)

    #CI for beta 0
    ci_b0_u = s_model.conf_int().T["const"][1]
    ci_b0_l = s_model.conf_int().T["const"][0]

    #CI for beta 1
    ci_b1_u = s_model.conf_int().T["x1"][1]
    ci_b1_l = s_model.conf_int().T["x1"][0]

    #Parameter estimates and p values
    b0_s = s_model.params["const"]
    p0_s = s_model.pvalues["const"]
    b1_s = s_model.params["x1"]





    p1_s = s_model.pvalues["x1"]

    #R2
    r2_s = s_model.rsquared

    #Store results
    s_results = {"W(%)":w_s,"CI.L.B0":ci_b0_l, "B0":b0_s, "CI.U.B0":ci_b0_u,"P(B0)":p0_s,
                            "CI.L.B1":ci_b1_l, "B1":b1_s, "CI.U.B1":ci_b1_u,"P(B1)":p1_s,
                 "R2":r2_s, "T":s_model.nobs}
    sequential_results.append(s_results)

#Store results as DF
sequential_results = pd.DataFrame(data = sequential_results)
sequential_results = pd.concat([pd.DataFrame(np.nan, index=range(2), columns=sequential_results.columns),
                                sequential_results], ignore_index=True)
#Prepend two rows of NaNs
if plot:
    #Plot rolling estimates, CIs, p-values
    fig, axs = plt.subplots(nrows=3, ncols=1, figsize=(15,10))

    #Beta 0 and 1
    for b in [0,1]:

        axs[b].plot(sequential_results.index, sequential_results[f"B{b}"], label=r"$\hat{\beta}$"+str(b))
        axs[b].fill_between(x=sequential_results.index,
                            y1=sequential_results[f"CI.L.B{b}"],
                            y2=sequential_results[f"CI.U.B{b}"], label="95% CI", alpha=0.2)

        #Overlay p-values
        axs[b].scatter(sequential_results.index,
                       np.where(sequential_results[f"P(B{b})"]<0.05, sequential_results[f"B{b}"], np.nan),
                       color="blue", label="p<5%", s=5)

        axs[b].set_title(s_model.model.exog_names[b])
        axs[b].grid()
        axs[b].legend()
        axs[b].axhline(y=0, linestyle="--", color="black", alpha=0.5)

    #Plot rolling R2
    axs[2].plot(sequential_results.index, sequential_results.R2, color="red")
    axs[2].set_title(r"$R^{2}$")
    axs[2].grid()

    #Set title, x label, and display
    plt.suptitle(f"Sequential OLS Results, T={len(Y)}")
```

```
        plt.xlabel(f"Window End\nNewey-West Std. Errors, {nw_lag} max. lags")
        plt.tight_layout()
        plt.show()

    return sequential_results

#Sequential OLS w/downweighting
def wls_downweighting(Y, X, lambda_d=0.95, a0=1.0, a1=1.0, plot=True):
    '''
    Set downweighting and regularization params.
    As a control, use 1.0, 0.0, 0.0. Final est should be OLS whole sample.
    Setting a0 > 0 imposes a minimum regularization.
    '''
    #Create dataset for non-recursive least squares
    nrls_data = pd.DataFrame()
    nrls_data["Y"] = Y
    nrls_data["x1"] = X

    #Compute regularization params
    nrls_data["a(t)"] = a0 + a1*(len(nrls_data) - np.arange(1,len(nrls_data)+1))

    #Compute downweighting factors per row
    nrls_data["W(t)"] = [[lambda_d**(t-i) for i in range(0, t+1)] for t in range(0, len(nrls_data))]

    #Compute NRLS beta
    def nrls_beta(X_t, W_t, Y_t, a_t):
        '''
        Args:
            X_t, pd.DataFrame: txk for k covariates
            W_t, list: length t list of diagonal elements
            Y_t, pd.DataFrame: tx1 response vector
            a_t, float: regularization parameter
        Returns:
            n_beta, array: vector of parameter estimates
        '''
        #Construct identity matrix dimension kxk
        I = np.identity(X_t.shape[1])

        #Ensure reposne and design inputs are matrices
        Y_t = np.array(Y_t)
        X_t = np.array(X_t)

        #Compute beta and return
        n_beta = np.linalg.inv(X_t.T @ W_t @ X_t + a_t * I) @ (X_t.T @ W_t @ Y_t)
        return n_beta




    #Variance/covariance matrix
    def cov_beta(X_t, W_t, Y_t, a_t, mse):

        #Construct identity matrix dimension kxk
        I_t = np.identity(X_t.shape[1])

        #Precision matrix
        P_t = np.linalg.inv(X_t.T @ W_t @ X_t + a_t*I_t)

        #Use closed form expression for vcov
        return mse*(P_t @ (X_t.T @ W_t.T @ W_t @ X_t) @ P_t)

    #1 covariate plus intercept, use t=3, 4...T. Note zero indexing.
    nrls_results = []
    for t in range(3, len(nrls_data)+1):

        #Subset data from 0 (inc.) to t-1 (inc.)
        sub_nrls_data = nrls_data.iloc[0:t,]

        #Set covariate matrix, include intercept
        X_tt = sm.add_constant(sub_nrls_data["x1"])
        X_tt = X_tt.values

        #Set response vector
        Y_tt = sub_nrls_data["Y"]
        Y_tt = Y_tt.values.reshape((len(Y_tt),1))

        #Set weight and regularization using values at t (row index t-1)
        W_tt = np.diag(sub_nrls_data.loc[t-1,"W(t)"])
        a_tt = sub_nrls_data.loc[t-1,"a(t)"]

        #Compute parameter estimates
        nbeta = nrls_beta(X_t=X_tt,W_t=W_tt, Y_t=Y_tt, a_t=a_tt)
        b0, b1 = float(nbeta[0]), float(nbeta[1])
        beta_t = nbeta.reshape((len(nbeta),1))

        #SSE and MSE
        sse = (Y_tt - X_tt @ beta_t).T @ W_tt @ (Y_tt - X_tt @ beta_t)
        mse = sse/(t-2)

        #estimated Variance/covariance matrix
        vcov_beta = cov_beta(X_t=X_tt, W_t=W_tt, Y_t=Y_tt, a_t=a_tt, mse=mse)

        #standard errors
```

```
        se_b0 = np.sqrt(vcov_beta[0][0])
        se_b1 = np.sqrt(vcov_beta[1][1])

        #CIs. 5% two sided, using t dist w/ T-p-1 DF.
        ci_b0_l = b0 - 1.9615*se_b0
        ci_b0_u = b0 + 1.9615*se_b0
        ci_b1_l = b1 - 1.9615*se_b1
        ci_b1_u = b1 + 1.9615*se_b1

        #p-value. (beta_hat - 0) / se ~ t(n-p-1)
        p_b0 = stats.t.sf(np.abs(b0/se_b0), len(Y_tt)-2-1)*2
        p_b1 = stats.t.sf(np.abs(b1/se_b1), len(Y_tt)-2-1)*2


        #Compute R2
        r2_t = r2_score(Y_tt, X_tt @ nbeta)

        #update results
        t_results = {"t":t,"CI.L.B0":ci_b0_l, "B0":b0, "CI.U.B0":ci_b0_u,"P(B0)":p_b0,
                         "CI.L.B1":ci_b1_l, "B1":b1, "CI.U.B1":ci_b1_u,"P(B1)":p_b1,
                 "R2":r2_t}
        nrls_results.append(t_results)

    #Store results as DF
    nrls_results = pd.DataFrame(data = nrls_results)
    nrls_results = pd.concat([pd.DataFrame(np.nan, index=range(2), columns=nrls_results.columns),
                              nrls_results], ignore_index=True)

    #Plot based upon flag
    if plot:

        #Plot downweighting and regularization schedule.
        fig, ax1 = plt.subplots(figsize=(15,10))

        #Decay weights on right axis in blue. Use weights at t=T.
        ax1.set_xlabel("Date")
        ax1.set_ylabel(r'$W_{t}$', color='blue')
        ax1.plot(nrls_data.index, nrls_data["W(t)"].values[-1], label="Decay Weights", color="blue")
        ax1.legend(loc="upper right")
        ax1.tick_params(axis='y',labelcolor='blue')

        #Down weight on left axis in red
        ax2 = ax1.twinx()
        ax2.set_ylabel(r'a(t)',color='red')
        ax2.plot(nrls_data.index, nrls_data["a(t)"], label="Regularization Parameter", color="red")




        ax2.legend(loc="upper left")
        ax2.tick_params(axis='y',labelcolor='red')

        #Set title and display
        plt.title("NRLS Downweighting and Regularization Schedule")
        plt.grid()
        plt.show()

        #Plot estimates, CIs, p-values
        fig, axs = plt.subplots(nrows=3, ncols=1, figsize=(15,10))

        #Beta 0 and 1
        for b in [0,1]:

            axs[b].plot(nrls_results.index, nrls_results[f"B{b}"], label=r"$\hat{\beta}$"+str(b))
            axs[b].fill_between(x=nrls_results.index,
                                y1=nrls_results[f"CI.L.B{b}"],
                                y2=nrls_results[f"CI.U.B{b}"], label="95% CI", alpha=0.2)

            #Overlay p-values
            axs[b].scatter(nrls_results.index,
                           np.where(nrls_results[f"P(B{b})"]<0.05, nrls_results[f"B{b}"], np.nan),
                           color="blue", label="p<5%", s=5)

            axs[b].set_title({0:"const",1:"x1"}[b])
            axs[b].grid()
            axs[b].legend()
            axs[b].axhline(y=0, linestyle="--", color="black", alpha=0.5)

        #Plot rolling R2
        axs[2].plot(nrls_results.index, nrls_results.R2, color="red")
        axs[2].set_title(r"$R^{2}$")
        axs[2].grid()

        #Set title, x label, and display
        plt.suptitle(f"Sequential Downweighting OLS Results, T={len(Y)}")
        plt.xlabel(f"Window End")
        plt.tight_layout()
        plt.show()

    #Return DF
    return nrls_results
```

Additional Approaches

I outline below additional approaches to estimating time-varying parameters. Preliminary testing showed these approaches need refinment and are computationally costly, hence their exclusion from the simulation exercise.

Background

Setup of problem. Time-varying parameters for contemporaneous time series regression. One covariate, parameters follow AR(1) processes.

$$\begin{aligned} Y_t &= a_t + b_t X_t + \epsilon_t & e_t &\sim \mathrm{WN}(0, \sigma_e^2) \\ a_t &= \mu_a + \phi_a a_{t-1} + w_{t,a} & w_{t,a} &\sim \mathrm{WN}(0, \sigma_a^2) \\ b_t &= \mu_b + \phi_b b_{t-1} + w_{t,b} & w_{t,b} &\sim \mathrm{WN}(0, \sigma_b^2) \end{aligned}$$

WN terms are serially and mutually uncorrelated. Assume further that $|\phi_a| < 1, |\phi_b| < 1$. Using recursive substitution:

$$\begin{aligned} a_t &= \mu_a + \phi_a \left(\mu_a + \phi_a a_{t-2} + w_{t-1,a}\right) + w_{t,a} \\ &= \mu_a + \phi_a \mu_a + \phi_a^2 \left(\mu_a + \phi_a a_{t-3} + w_{t-2,a}\right) + \phi_a w_{t-1,a} + w_{t,a} \\ &= \mu_a + \phi_a \mu_a + \phi_a^2 \mu_a + \phi_a^3 \left(\mu_a + \phi_a a_{t-4} + w_{t-3,a}\right) + \phi_a^2 w_{t-2,a} + \phi_a w_{t-1,a} + w_{t,a} \\ &\vdots \\ &= \mu_a \left(\sum_{j=0}^{t-1} \phi_a^j\right) + \phi_a^t a_0 + \sum_{j=0}^{t-1} \phi_a^j w_{t-j,a} \\ b_t &= \mu_b \left(\sum_{j=0}^{t-1} \phi_b^j\right) + \phi_b^t b_0 + \sum_{j=0}^{t-1} \phi_b^j w_{t-j,b} \end{aligned}$$

The long run expected value of the parameters (as $t- > \infty$):

$$\begin{aligned} E(a_t) &= \frac{\mu_a}{1-\phi_a} + \sum_{j=0}^{t-1} \phi_a^j E(w_{t-j,a}) \\ &= \frac{\mu_a}{1-\phi_a} \\ E(b_t) &= \frac{\mu_b}{1-\phi_b} + \sum_{j=0}^{t-1} \phi_b^j E(w_{t-j,b}) \\ &= \frac{\mu_b}{1-\phi_b} \end{aligned}$$

For simulation purposes, set ϕ_a, ϕ_b to nearly, but not exactly 1. For large T, a value like 0.5 implies quick convergence. The goal is to simulate close to RWs and then RWs for the parameters. Simulate using drift values of 0.

Substitute into Y_t and rearrange

$$\begin{aligned} Y_t &= a_t + b_t X_t + \epsilon_t \\ &= \left[\mu_a \left(\sum_{j=0}^{t-1} \phi_a^j\right) + \phi_a^t a_0 + \sum_{j=0}^{t-1} \phi_a^j w_{t-j,a}\right] + \left[\mu_b \left(\sum_{j=0}^{t-1} \phi_b^j\right) + \phi_b^t b_0 + \sum_{j=0}^{t-1} \phi_b^j w_{t-j,b}\right] X_t + \epsilon_t \\ &= \left[\mu_a \left(\sum_{j=0}^{t-1} \phi_a^j\right) + \phi_a^t a_0\right] + \left[\mu_b \left(\sum_{j=0}^{t-1} \phi_b^j\right) + \phi_b^t b_0 + \sum_{j=0}^{t-1} \phi_b^j w_{t-j,b}\right] X_t + \left(\epsilon_t + \sum_{j=0}^{t-1} \phi_a^j w_{t-j,a}\right) \\ &= \left[\frac{\mu_a(1-\phi_a^t)}{1-\phi_a} + \phi_a^t a_0\right] + \left[\frac{\mu_b(1-\phi_b^t)}{1-\phi_b} + \phi_b^t b_0 + \sum_{j=0}^{t-1} \phi_b^j w_{t-j,b}\right] X_t + \left(\epsilon_t + \sum_{j=0}^{t-1} \phi_a^j w_{t-j,a}\right) \end{aligned}$$

The complication is the product of X_t and a weighted average of stochastic white noise terms w_t, where the product uses the entire history of noise terms and exponentially decaying weights.

Rolling OLS (Custom Kernel)

Aside from CV for h, the value added is a custom kernel function that mimics the AR(1) properties of the coefficients. Kernel choice can impact smoothness of estimates (Gaussian should produce smoother estimates than rectangular, all else equal). Let $K(x) = \mathbf{1}(x \geq 0) \cdot |\phi|^x$. This is not symmetric, but evaluates to 0 when $t \geq i$ and operates more like rolling/sequential: observations to right of t (temporally) get 0 weight. It reflects the distinction between a filter and smoother. It must be that $0 < \phi < 1$. As Phi approaches 0, earlier observations (farther from t) get no weight, as phi approaches 1, closer to equal weighting. At phi = 1, we just get sequential OLS (expanding).

Sequential OLS

Let $t- > \infty$, and assume X_t is finite such that the product of X_t and a decaying weight is 0 at the limit. Regardless of AR(1) coefficient values (aside from exactly 1), it is the case that:

$$
\begin{aligned}
Y_t &= \frac{\mu_a}{1-\phi_a} + \left[\frac{\mu_b}{1-\phi_b} + \sum_{j=0}^{t-1} \phi_b^j w_{t-j,b}\right] X_t + \left(\epsilon_t + \sum_{j=0}^{t-1} \phi_a^j w_{t-j,a}\right) \\
E(Y_t|X_t) &= \frac{\mu_a}{1-\phi_a} + \frac{\mu_b}{1-\phi_b} X_t + \sum_{j=0}^{t-1} \phi_b^j E(w_{t-j,b}|X_t) + E\left(\epsilon_t + \sum_{j=0}^{t-1} \phi_a^j w_{t-j,a}\right) \\
&= \frac{\mu_a}{1-\phi_a} + \frac{\mu_b}{1-\phi_b} X_t
\end{aligned}
$$

The coefficients are just the long run expected values of the autoregressive processes. Note a_t and b_t cannot be RWs to use the geometric sum.

Sequential WLS (Downweighting)

If $-1 < \phi_a, \phi_b < 1$, the processes for a(t) and b(t) are stationary and should quickly converge to their long run means. Expanding the OLS window sequentially should produce 'better' estimates of a(t) and b(t) for no other reason that they eventually become constants. If ϕ_a or ϕ_b equal 1, a(t) and b(t) are random walks and longer time series don't facilitate estimation because a(t) and b(t) do not have time-invariant sample moments to converge to.

I use a variant of penalized WLS which downweights older observations and shrinks parameter estimates to prevent overfitting. The regularization parameter is a function of t, and decreases in magnitude as t increases. For small t, meaning OLS with a few observations, regularization is stronger. This filtering approach is like sequential OLS, in that it only uses past data (in contrast, kernel regression is akin to smoothing in that it access all observations). Unlike rolling OLS, which assigns equal weight to all observations within the window, this procedure assigns some decaying weight to all prior observations and 0 weight to all future observations.

Setup:

For t = 1...T, let:

- y_i be a scalar response
- Y_t a $tx1$ vector of responses including $y_1 \ldots y_t$
- X_t a txk matrix of covariates.
- There are k parameters, contained in a $kx1$ vector β_t.
- Assume the relationship is linear, for $\epsilon_t \sim \mathrm{WN}(0, \sigma_\epsilon^2 I_t)$:

$$
Y_t = X_t \beta_t + \epsilon_t
$$

- W_t be a diagonal matrix with elements $\lambda^{t-1}, \lambda^{t-2}, \ldots, \lambda^{t-(t-1)}, \lambda^{t-t}$ for $0 \leq \lambda \leq 1$. The dimensions of W_t vary with t:

$$
W_t = \begin{bmatrix} \lambda^{t-1} & \cdots & & 0 \\ & \lambda^{t-2} & & \\ \vdots & & \ddots & \vdots \\ & & \lambda & \\ 0 & & & 1 \end{bmatrix}_{t \times t}
$$

As t increases for fraction λ, the first weight goes to 0. Weights decay to 0. Let $w_t = W_t^{0.5}$, note that $w_t' w_t = W_t$.

For estimation purposes, iterate from $t = 3 \ldots T$, expanding the vectors/matrices to include t observations (sequentially adding data points). Parameters are now time-varying because observations get down-weighted based on age. Define the weighted loss function as:

$$
\begin{aligned}
L_t(\beta_t) &= ||w_t \epsilon_t||_2^2 + \alpha_t ||\beta_t||_2^2 \\
&= (w_t \epsilon_t)' (w_t \epsilon_t) + \alpha_t \beta_t' \beta_t \\
&= \epsilon_t' w_t' w_t \epsilon_t + \alpha_t \beta_t' \beta_t \\
&= (Y_t - X_t \beta_t)' W_t (Y_t - X_t \beta_t) + \alpha_t \beta_t' \beta_t
\end{aligned}
$$

where I include a time-varying regularization term α_t to prevent overfitting. For small t, the limited sample size makes overfitting an acute concern, but for large t overfitting isn't as much of a concern. At $\alpha_t = 0$, the penalty term drops out. As α_t goes to ∞, the vector θ_t is pushed towards $\vec{0}$.

Solving:

$$
\begin{aligned}
\min_\beta L_t(\beta) &= (Y_t - X_t \beta)' W_t (Y_t - X_t \beta) + \alpha_t \beta' \beta \\
\nabla_\beta L_t(\beta) &= 0 \\
0 &= \nabla_\beta (Y_t' W_t - \beta' X_t' W_t)(Y_t - X_t \beta) + \alpha_t \beta' \beta \\
&= \nabla_\beta \left(Y_t' W_t Y_t - \beta' X_t' W_t Y_t - Y_t' W_t X_t \beta + \beta' X_t' W_t X_t \beta + \alpha_t \beta' \beta\right) \\
0 &= -X_t' W_t Y_t - X_t' W_t' Y_t + 2 X_t' W_t X_t \beta + 2\alpha_t I_t \beta \\
2X_t' W_t Y_t &= 2(X_t' W_t X_t + \alpha_t I_t)\beta \\
\beta^* &= (X_t' W_t X_t + \alpha_t I_t)^{-1} \left(X_t' W_t Y_t\right)
\end{aligned}
$$

As expected, at $W_t = \sigma_\epsilon^2 I_t$ and $\alpha = 0$, the optimal estimator is the OLS estimator.

Experiment with different forms for α_t. The simplest is a linear time trend:

$$\alpha_t = a_0 + a_1(T - t), a_0, a_1 \geq 0$$

for complete sample size T, so regularization strength decreases as $t \to T$. Standard errors, confidence intervals, and p-values follow from the variance of the estimator:

$$\begin{aligned}
\beta^* &= \left(X_t' W_t X_t + \alpha_t I_t\right)^{-1} \left(X_t' W_t Y_t\right) \\
\mathrm{Var}(\beta^* | X_t) &= \left(X_t' W_t X_t + \alpha_t I_t\right)^{-1} \mathrm{Var}\left(X_t' W_t X_t \beta_t + X_t' W_t \epsilon_t | X_t\right) \left(X_t' W_t X_t + \alpha_t I_t\right)^{-1} \\
&= \left(X_t' W_t X_t + \alpha_t I_t\right)^{-1} X_t' W_t \sigma_\epsilon^2 I_t W_t' X_t \left(X_t' W_t X_t + \alpha_t I_t\right)^{-1} \\
&= \sigma_\epsilon^2 \left(X_t' W_t X_t + \alpha_t I_t\right)^{-1} X_t' W_t W_t' X_t \left(X_t' W_t X_t + \alpha_t I_t\right)^{-1}
\end{aligned}$$

To estimate the variance σ_ϵ^2, I use the weighted sum of squared errors divided by the degrees of freedom, where I computed the weighted SSE using the time varying parameter estimates and weights. The estimate of σ_ϵ^2 is therefore also time-varying:

$$\begin{aligned}
\hat{\sigma}_{\epsilon,t}^2 &= \frac{(Y_t - X_t \hat{\beta}_t)' W_t (Y_t - X_t \hat{\beta}_t)}{t - 2 - 1} \\
\widehat{\mathrm{Cov}}(\beta_t) &= \hat{\sigma}_{\epsilon,t}^2 \left(X_t' W_t X_t + a_t I_t\right)^{-1} X_t' W_t' W_t X_t \left(X_t' W_t X_t + a_t I_t\right)^{-1}
\end{aligned}$$

Estimate of error term's variance decreases mechanically as t increases. Standard errors are the square root of the diagonal elements of the estimated covariance matrix. I construct CIs using the form:

$$\begin{aligned}
\hat{\beta}_{0,t} &\pm 1.96 \cdot \mathrm{SE}(\hat{\beta}_{0,t}) \\
\hat{\beta}_{1,t} &\pm 1.96 \cdot \mathrm{SE}(\hat{\beta}_{1,t})
\end{aligned}$$

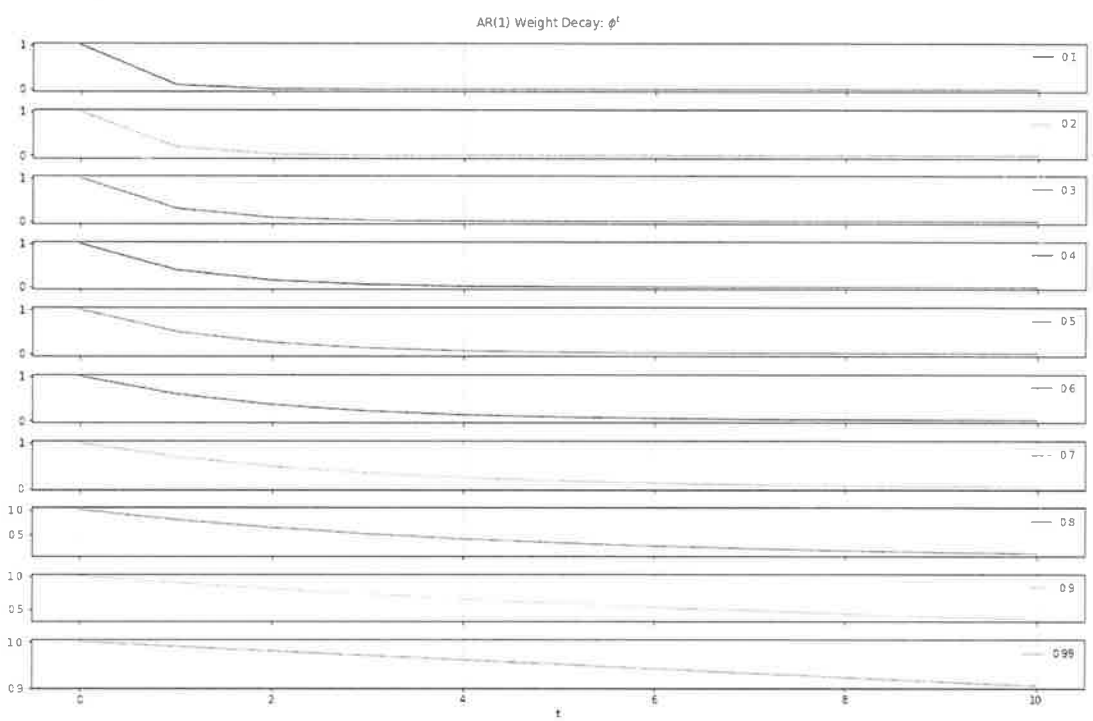
Given the large sample size, the estimate of the degrees of freedom, use of a 1.96 CV corresponding to a 95% CI, the absence of controls for heteroskedasticity and autocorrelation in ϵ, is tolerable.

I report R2 per window. At each t, I compute $\hat{Y}_t$ and associated R2.

Get an intuitive sense of weight decay schedule for the one sided ACF 'Kernel':

```
#Demonstrate weight schedule
phi_weight = pd.DataFrame(data=[np.arange(0,10+1)], index=["t"]).T
for phi in list(np.arange(0.10,0.90+0.10, 0.10))+[0.99]:
    phi_weight[f"{np.round(phi,3)}"] = phi**phi_weight["t"]
```

```
phi_weight = phi_weight.set_index("t")
phi_weight.plot(subplots=True, figsize=(15,10), grid=True)
plt.suptitle("AR(1) Weight Decay: "+r'$\phi^{t}$')
plt.tight_layout()
plt.show()
```



Simulation Testing

Simulate from this process:

$$
\begin{aligned}
Y_t &= a_t + b_t X_t + \epsilon_t \\
a_t &= \mu_a + \phi_a a_{t-1} + w_{t,a} \\
b_t &= \mu_b + \phi_b b_{t-1} + w_{t,b} \\
[X_t\ e_t\ w_{t,a}\ w_{t,b}]' &\overset{\text{i.i.d.}}{\sim} \text{MVN}(\mathbf{0}, I_4)
\end{aligned}
$$

Set $\phi_a, \phi_b = 1.0$ to get random walks. X_t is a N(0,1) draw. Use N=1000, T=100. Compare Gaussian (two-sided) kernel to one-sided ACF and one-sided Gaussian.

Gaussian:

$$
\begin{aligned}
\text{Gaussian: } K(x) &= Ce^{-0.5x^2} \\
\text{One-Sided: } K(x) &= \mathbf{1}\{x \le 0\} \cdot Ce^{-0.5x^2} \\
\text{ACF: } K(x) &= \mathbf{1}\{x \le 0\} \cdot |\widehat{\phi}|^x
\end{aligned}
$$

```
#Set seed for reproducability
np.random.seed(0)

#Call simulation function, use defaults and X(t) ~ N(0,1) draws
sim_T = 150
sim_X = np.random.normal(0,1,sim_T)
tvp_sim, a_sim, b_sim = sim_tvp(sim_X, T=len(sim_X), phi_a = 1.0, phi_b=1.0, n=1000)
tvp_sim = pd.DataFrame(data = tvp_sim)
tvp_sim.index.name = "Simulation"

#Set last l
last_T = sim_T-50
t0 = time.time()
n_test = tvp_sim.shape[0]
#Aggregate results into df
results_list = []

#Two types of Gaussian kernels: one sided and two sided
def two_sided_gk(x):
    return np.exp(-0.5*(x**2))

def one_sided_gk(x):
    return np.where(x>=0, np.exp(-0.5*(x**2)), 0)
```

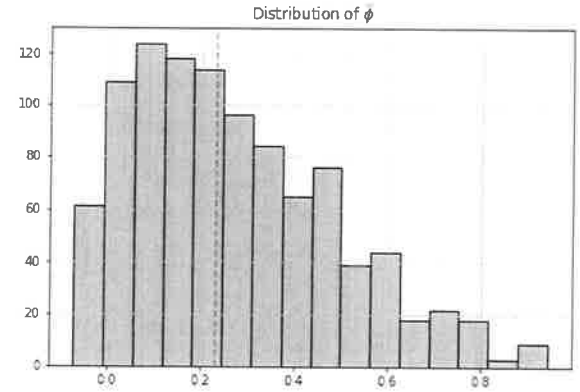
```
#For each sim, call estimation methods, compute errors
for i in range(n_test):

    #Set Yt as a row of tvp_sim
    Yt = tvp_sim.loc[i,:]

    #Set at and bt. Use last 100.
    at = a_sim[i,-last_T:]
    bt = b_sim[i,-last_T:]

    #Use standard Gaussian kernel.
    gaussian_kernel_results = kernel_reg_sim(Yt, sim_X, kernel=two_sided_gk)

    #One sided Gaussian
    one_sided_gaussian_kernel_results = kernel_reg_sim(Yt, sim_X, kernel=one_sided_gk)

    #Now use ACF inspired 'kernel'. Compute whole sample AR(1) Phi:
    ws_phi = sm.OLS(Yt[1:].values,sm.add_constant(Yt[:-1].values)).fit().params[1]
    acf_kernel_results = kernel_reg_sim(Yt, sim_X, kernel=lambda x: np.where(x<=0, 0, np.abs(ws_phi)**(x)))

    #The ACF kernel is one sided. Take last 100 obs.
    acf_kernel_results = acf_kernel_results.tail(last_T)
    gaussian_kernel_results = gaussian_kernel_results.tail(last_T)
    one_sided_gaussian_kernel_results = one_sided_gaussian_kernel_results.tail(last_T)

    #Gaussian R2, RMSE, ACF R2, RMSE
    results = {}
    results["Phi"] = ws_phi
    for results_df, label in zip([gaussian_kernel_results, one_sided_gaussian_kernel_results, acf_kernel_results],
                                 ["Gaussian","One-Sided Gaussian","ACF"]):
        for variable, actual in zip(["B0","B1"],[at,bt]):
            results[f"{label} {variable} R2"] = r2_score(results_df[variable], actual)
            #results[f"{label} {variable} RMSE"] = mean_squared_error(results_df[variable], actual)**0.5
    results_list.append(results)

#Check execution time
t1 = time.time()
print(f"{np.round(t1-t0,2)}s")

#Create dataframe from list of dictionaries, sort
sim_results = pd.DataFrame(results_list)
sim_results = sim_results[sorted(sim_results.columns)]
sim_results.index.name="Sim."
```

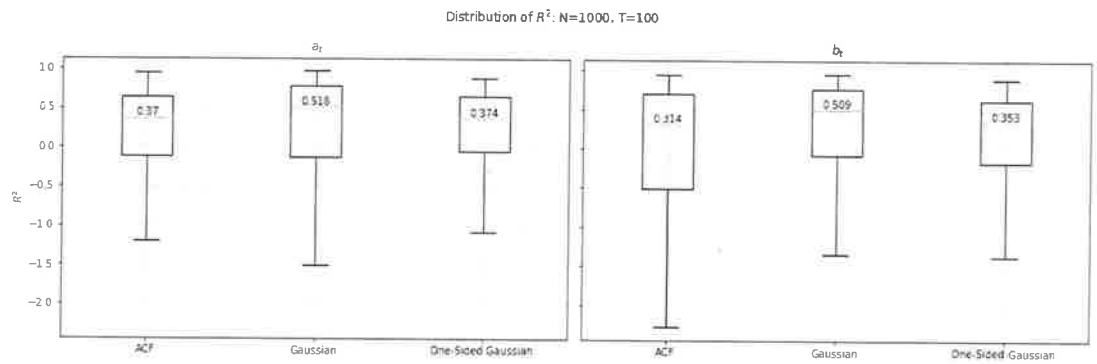
```
#Error summary stats
print(f"\nSummmary Stats, N={n_test},T={last_T}")
sim_summ_stats = sim_results.describe(percentiles=[0.01,0.25,0.50,0.75,0.99])
display(sim_summ_stats.round(3))

#Phi Dist
try:
    plt.figure(figsize=(7.5,5.0))
    plt.grid()
    plt.title("Distribution of "+r"$\widehat{\phi}$")
    plt.axvline(x=np.median(sim_results[["Phi"]].values), linestyle="--")
    sns.histplot(data=sim_results[["Phi"]].values, kde=True)
except ValueError:
    pass

#Subplots
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=(15,5), sharex=False, sharey=True)

#B0
axs[0].boxplot(sim_results[[f"ACF B0 R2",f"Gaussian B0 R2",f"One-Sided Gaussian B0 R2"]], vert=True, sym="")
axs[0].set_ylabel(r"$R^{2}$")
axs[0].set_xticklabels(['ACF', 'Gaussian',"One-Sided Gaussian"])
axs[0].set_title(r"$a_{t}$")
axs[0].grid()
axs[0].text(1, sim_summ_stats["ACF B0 R2"]["50%"],
            str(sim_summ_stats["ACF B0 R2"]["50%"].round(3)),ha='center', va='bottom')
axs[0].text(2, sim_summ_stats["Gaussian B0 R2"]["50%"],
            str(sim_summ_stats["Gaussian B0 R2"]["50%"].round(3)),ha='center', va='bottom')
axs[0].text(3, sim_summ_stats["One-Sided Gaussian B0 R2"]["50%"],
            str(sim_summ_stats["One-Sided Gaussian B0 R2"]["50%"].round(3)),ha='center', va='bottom')


#B1
axs[1].boxplot(sim_results[[f"ACF B1 R2",f"Gaussian B1 R2",f"One-Sided Gaussian B1 R2"]], vert=True, sym="")
axs[1].set_xticklabels(['ACF', 'Gaussian',"One-Sided Gaussian"])
axs[1].set_title(r"$b_{t}$")
axs[1].grid()
axs[1].text(1, sim_summ_stats["ACF B1 R2"]["50%"],
            str(sim_summ_stats["ACF B1 R2"]["50%"].round(3)),ha='center', va='bottom')
axs[1].text(2, sim_summ_stats["Gaussian B1 R2"]["50%"],
            str(sim_summ_stats["Gaussian B1 R2"]["50%"].round(3)),ha='center', va='bottom')
axs[1].text(3, sim_summ_stats["One-Sided Gaussian B1 R2"]["50%"],
            str(sim_summ_stats["One-Sided Gaussian B1 R2"]["50%"].round(3)),ha='center', va='bottom')




#Set title, display
plt.suptitle("Distribution of "+"$R^{2}$: "+f"N={n_test}, T={last_T}")
plt.tight_layout()
plt.show()
```

73.26s

Summmary Stats, N=1000,T=100

	ACF B0 R2	ACF B1 R2	Gaussian B0 R2	Gaussian B1 R2	One-Sided Gaussian B0 R2	One-Sided Gaussian B1 R2	Phi
count	1000.000	1000.000	1000.000	1000.000	1000.000	1000.000	1000.000
mean	0.024	-0.689	-0.075	-0.085	0.165	0.097	0.269
std	1.149	4.441	1.875	1.913	0.746	0.789	0.217
min	-12.760	-94.273	-27.778	-18.956	-5.280	-5.409	-0.071
1%	-4.515	-13.380	-7.506	-10.212	-2.897	-3.115	-0.054
25%	-0.110	-0.509	-0.135	-0.077	-0.049	-0.173	0.093
50%	0.370	0.314	0.518	0.509	0.374	0.353	0.232
75%	0.640	0.699	0.787	0.771	0.653	0.628	0.415
99%	0.892	0.926	0.950	0.939	0.857	0.872	0.821
max	0.947	0.949	0.975	0.971	0.884	0.899	0.943





Results

- Unlike the first simulation exercise, in which a_t was near zero, simulated a_t is a pure random walk just like b_t. The processes are identical, the simulated parameter vectors can even be interchanged.

- The ACF window function achieves similar performance to the one-sided Gaussian. Both lag the conventional (two-sided) Gaussian kernel. The difference is about 14% R^2 at the median, but for b_t the dispersion for the ACF window R^2 values is noticeably larger.

- Interpret the difference in summary statistics between Gaussian and ACF or Gaussian and one-sided Gaussian as the reduction in parameter recovery when using smoothing versus filtering methods. Some loss in explanatory power is to be expected. These results provide a baseline to answer the question of compared to what.

- Despite having random walk parameters and IID draws for covariate values, the simulated process Y_t is nowhere close to a random walk. The persistence, on average, is quite low, because Y_t is a combination of stochastic processes. X_t is stationary, a_t and b_t are random walks, and e_t is stationary.

General observations:

- Rolling OLS: What t index does rolling OLS estimate? The middle of the window coheres with the framing of kernel regression, but this entails using data after the midpoint to estimate parameters at the midpoint, which is problematic for forecasting. If you use the end of the window, rolling OLS is akin to sequential OLS but drops trailing observations.

- Kernel Regression R2 needs to be examined carefully. Conventional rolling OLS reports R2 over the window, so there are T-W+1 R2 figures for increments of 1 and window size W. Kernel regression reports R2 over the entire period R2 using in sample predictions from a kernel function centered at specific t.

- Sequential OLS is useful to show the marginal effects of each additional data point on the regression's goodness of fit and parameter estimates. It can identify hinge points in the model, such as breaks or shocks that pull the regression slope in one direction.